

ปฏิบัติการครั้งที่ 4

เรื่อง Selectors & Asyncio

เวลาทำปฏิบัติการ

150 นาที

ผลลัพธ์การเรียนรู้ (Learning Outcomes)

เมื่อปฏิบัติการเสร็จสิ้น นักศึกษาจะสามารถ:

1. อธิบายความต่างระหว่าง blocking และ non-blocking socket ได้
2. ใช้ selectors.DefaultSelector() สร้าง event-driven server ที่รับ client หลายคนพร้อมกันได้ใน 1 thread
3. เขียน asyncio server ด้วย async def และ await ได้ถูกตำแหน่ง
4. เปรียบเทียบ Threading, Selectors, asyncio และเลือกใช้งานได้เหมาะสม
5. จัดการ shared state ใน asyncio โดยไม่ต้องใช้ Lock

เครื่องมือที่ใช้ในปฏิบัติการ

1. Python version 3.x up
2. Visual studio code
3. Terminal

ไฟล์ที่ต้องส่ง

task1_selector_echo.py, task2_selector_broadcast.py, task3_asyncio_echo.py,
task4_asyncio_chat.py



แจ้งเตือนก่อนทำปฏิบัติการ

กติกาสำคัญของ Lab 04: ห้ามใช้ threading หรือ ThreadPoolExecutor ในทุก Task — Task 1-2 ใช้ selectors เท่านั้น, Task 3-4 ใช้ asyncio เท่านั้น

แบบฝึกหัด

Selector Echo Server

1

เวลา: 40 นาที | Skill: selectors, non-blocking socket, callback

โจทย์: เขียน TCP Echo Server แบบ event-driven ด้วย selectors โดย server ต้องรองรับ client หลายตัวพร้อมกันใน 1 thread (ห้ามใช้ thread)

ตัวอย่างโค้ดที่เตรียมไว้ให้

Skeleton — task1_selector_echo.py

```
# task1_selector_echo.py
import socket
import selectors

HOST = '0.0.0.0'
PORT = 9000

sel = selectors.DefaultSelector()

def accept(sock, mask):
    conn, addr = sock.accept()
    print(f'[+] Connected: {addr}')
    # TODO: setblocking(False) ให้ conn
    # TODO: register conn เข้า selector ด้วย EVENT_READ → callback read

def read(conn, mask):
    try:
        data = conn.recv(1024)
        if data:
            # TODO: echo data กลับไปหา client
            pass
        else:
            # TODO: unregister + close
            pass
    except Exception:
        # TODO: handle error - unregister + close
        pass

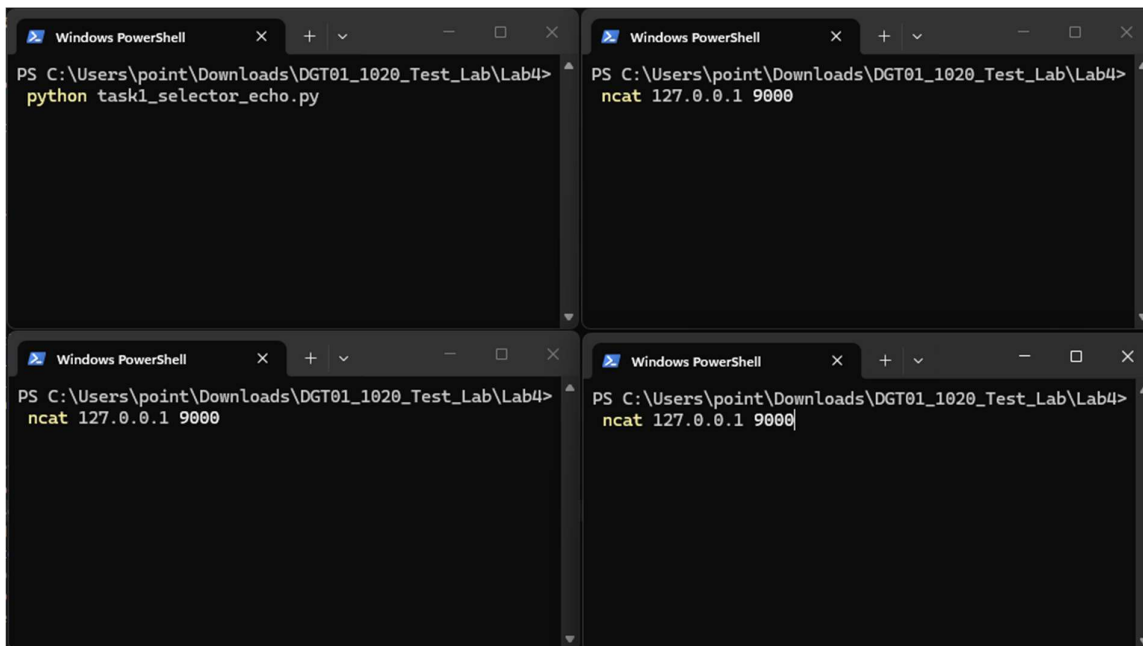
# Setup server socket
server = socket.socket()
server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
server.bind((HOST, PORT))
server.listen()
server.setblocking(False)
```

```
sel.register(server, selectors.EVENT_READ, accept)
print(f'[SERVER] Selector Echo running on {HOST}:{PORT}')

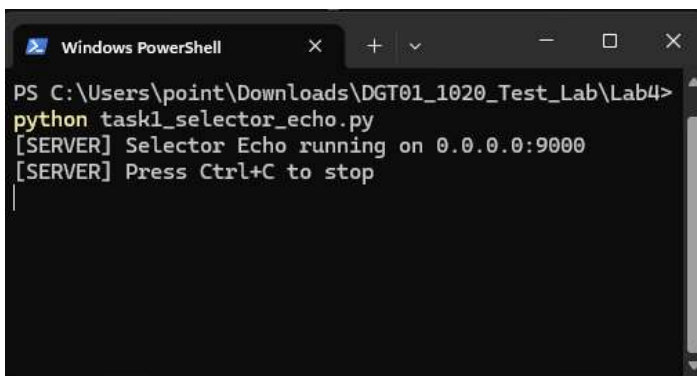
# Event loop
while True:
    for key, mask in sel.select():
        callback = key.data
        callback(key.fileobj, mask)
```

ผลลัพธ์ที่คาดหวัง: โดยก่อนการรัน - เปิด Terminal 4 หน้าต่าง โดยเตรียมการรันไฟล์ต่างๆ ตามรูป

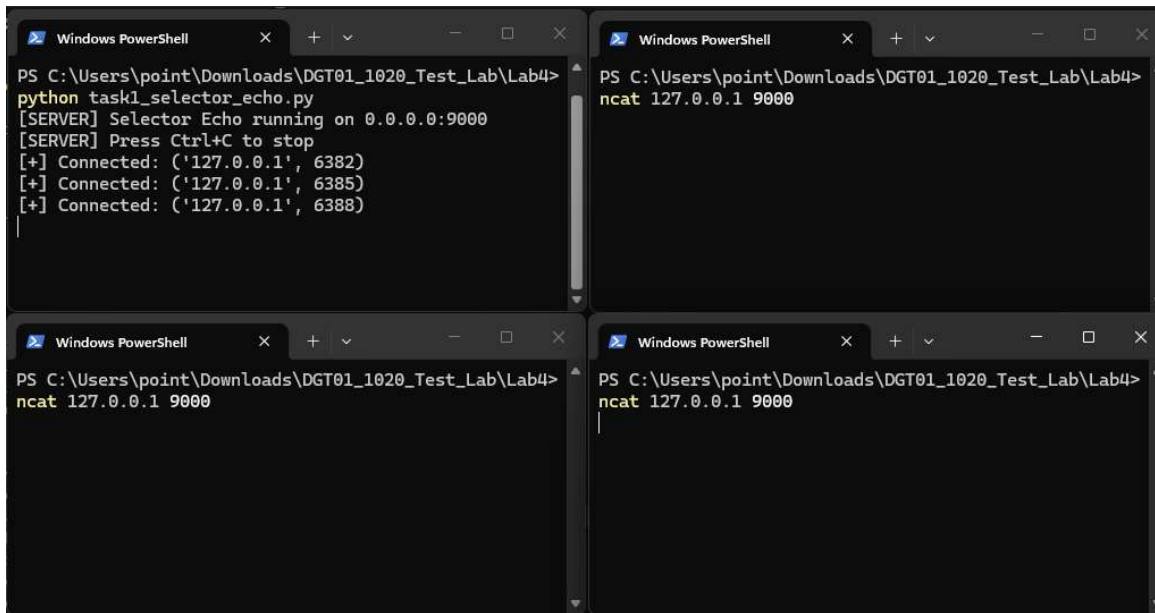
***สำหรับคำสั่ง ncat ที่ปรากฏอยู่ใน Terminal ตามรูปที่แนบดังกล่าว เป็นการแนะนำหากไม่ยากเขียนไฟล์ Client และใช้ทดแทนได้ดีกว่า Telnet สามารถติดตั้งผ่านโปรแกรม nmap ได้โดยกดลิงค์ที่แนบดังนี้ [nmap-7.99-setup.exe](#) เมื่อติดตั้งแล้วให้ปิด-เปิดโปรแกรม Terminal หรือ VScode ใหม่ เพื่อให้โปรแกรมเรียกใช้ฟังก์ชันได้



ผลลัพธ์เมื่อทดลองรัน Server

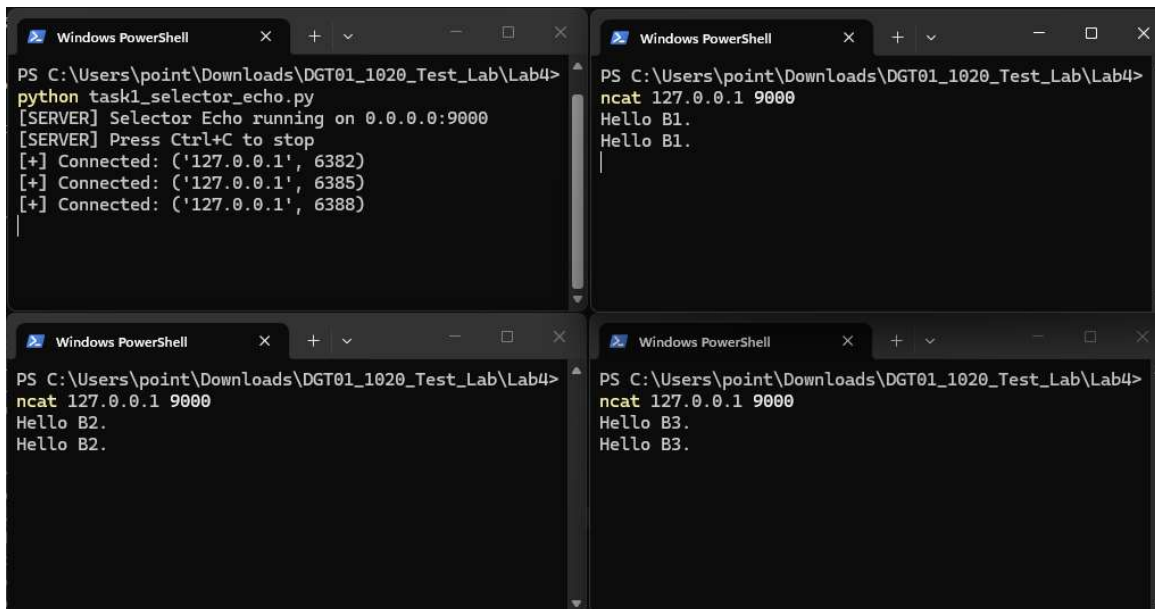


ผลลัพธ์เมื่อทดลองรัน Server แล้วใช้คำสั่ง ncat 127.0.0.1 9000 ที่ Terminal ทั้ง 4 หน้าต่าง ที่ต้องการรัน Chat Client



The image shows four terminal windows arranged in a 2x2 grid. The top-left window shows the server script being executed: `python task1_selector_echo.py`. The output indicates the server is running on 0.0.0.0:9000 and has successfully connected to three clients at ports 6382, 6385, and 6388. The top-right window shows the command `ncat 127.0.0.1 9000` being entered in a terminal. The bottom-left window shows the same `ncat` command being entered. The bottom-right window shows the same `ncat` command being entered.

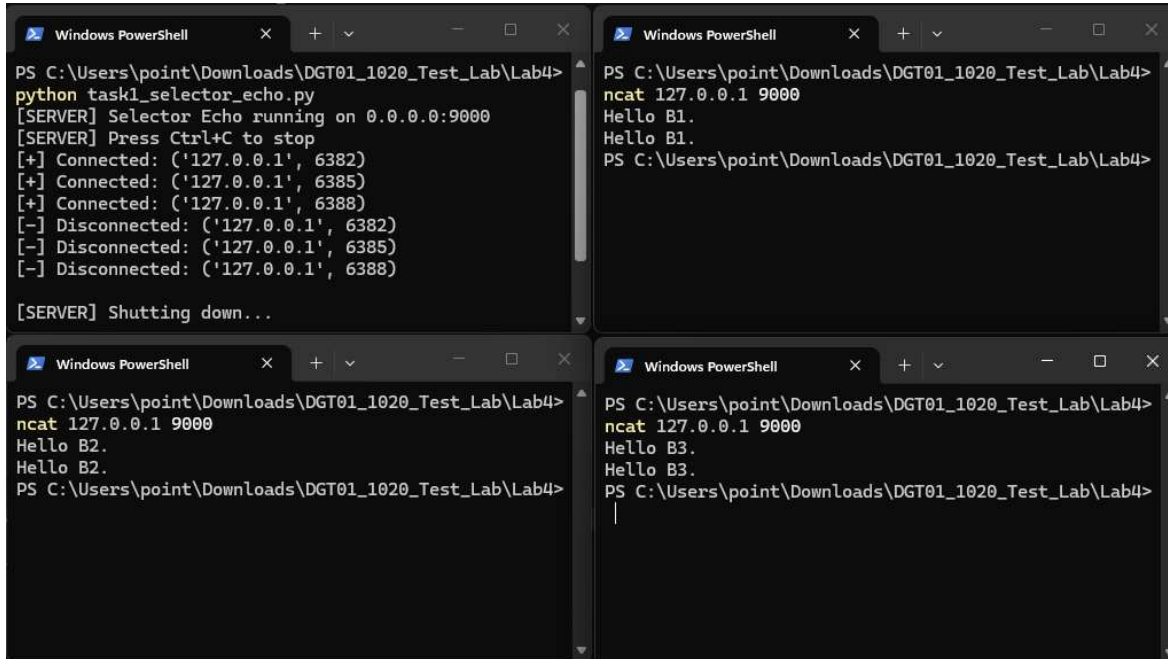
ผลลัพธ์เมื่อทดลองพิมพ์ข้อความใน Client บน Terminal อีก 3 หน้าต่าง โดยจะตอบกลับตามที่เราป้อนข้อความเข้าไป โดย Server



The image shows the same four terminal windows as before, but now with messages being exchanged. The top-left window (server) shows the same initial output. The top-right window shows the client sending "Hello B1." and receiving "Hello B1." in response. The bottom-left window shows the client sending "Hello B2." and receiving "Hello B2." in response. The bottom-right window shows the client sending "Hello B3." and receiving "Hello B3." in response.

มีตัวอย่างผลลัพธ์ต่อในหน้าถัดไป

ผลลัพธ์เมื่อทำการกด Ctrl + C เพื่อหยุดการทำงานทุกคำสั่งทุก Terminal



The image shows four terminal windows arranged in a 2x2 grid, all running Windows PowerShell. The top-left window shows the execution of a Python script named `task1_selector_echo.py`. The script outputs: `[SERVER] Selector Echo running on 0.0.0.0:9000`, `[SERVER] Press Ctrl+C to stop`, and then a series of connection and disconnection messages for IP addresses 127.0.0.1 on ports 6382, 6385, and 6388. It ends with `[SERVER] Shutting down...`. The top-right window shows the same script being run, but it is terminated with Ctrl+C, resulting in `ncat 127.0.0.1 9000` and `Hello B1.` being printed twice before the prompt returns. The bottom-left window shows the script being run, terminated with Ctrl+C, and `Hello B2.` being printed twice. The bottom-right window shows the script being run, terminated with Ctrl+C, and `Hello B3.` being printed twice.

Hint: ลืม `setblocking(False)` บ่อยที่สุด — ทั้ง `server socket` (ก่อน `register`) และ `conn` ที่ได้จาก `accept()` ต้องใส่ทั้งคู่ ถ้าลืม `event loop` จะ hang

เงื่อนไขผ่าน

- ✓ ใช้ `selectors.DefaultSelector()`
- ✓ `server socket` ต้อง `setblocking(False)`
- ✓ `client socket` ที่ได้จาก `accept()` ต้อง `setblocking(False)`
- ✓ มี `callback` อย่างน้อย 2 ตัว: `accept(sock, mask)` และ `read(conn, mask)`
- ✓ echo ข้อความกลับไปหา client
- ✓ เมื่อ client disconnect ต้อง `unregister()` และ `close()`
- ✓ ห้ามใช้ `threading` หรือ `ThreadPoolExecutor`

แบบฝึกหัด

Selector Broadcast Server

2

เวลา: 40 นาที | Skill: shared state, broadcast, error handling

โจทย์: ต่อยอดจาก Task 1 ให้ server broadcast ข้อความจาก client หนึ่งไปหา client ทุกคน โดยยังใช้ selectors และไม่ใช่ thread

ตัวอย่างโค้ดที่เตรียมไว้ให้

Skeleton — task2_selector_broadcast.py

```
# task2_selector_broadcast.py
import socket
import selectors

HOST = '0.0.0.0'
PORT = 9001

sel = selectors.DefaultSelector()
clients = set()    # เก็บ client socket ทั้งหมด

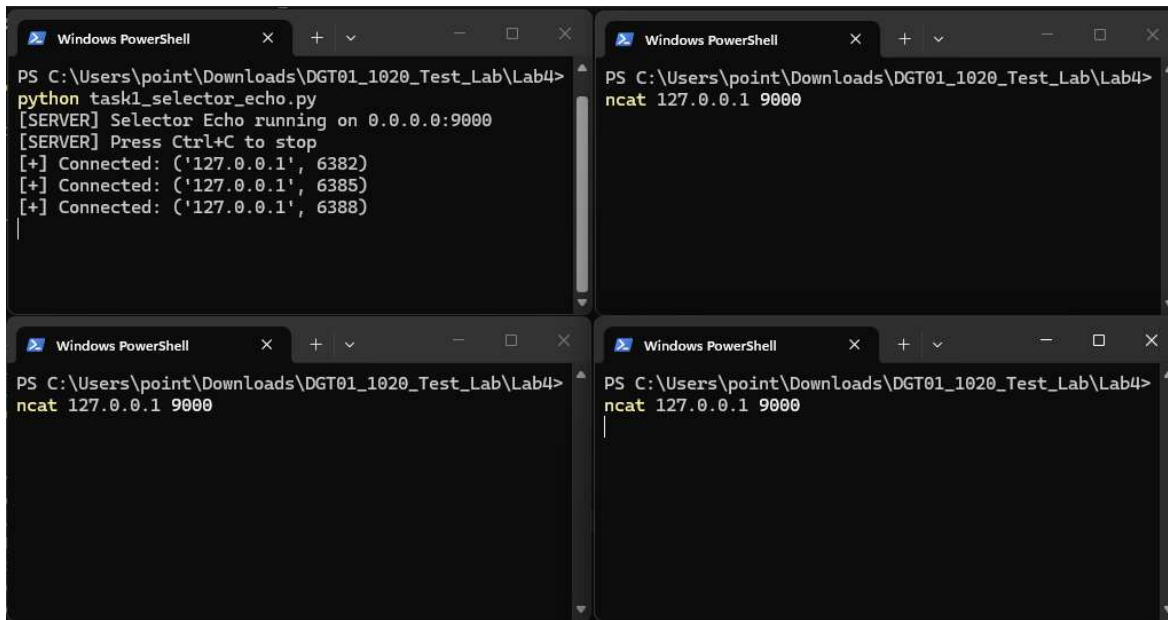
def accept(sock, mask):
    conn, addr = sock.accept()
    print(f'[+] Connected: {addr}')
    conn.setblocking(False)
    sel.register(conn, selectors.EVENT_READ, read)
    # TODO: เพิ่ม conn เข้า clients

def read(conn, mask):
    try:
        data = conn.recv(1024)
        if data:
            addr = conn.getpeername()
            msg = f'{addr}: {data.decode()}'
            # TODO: broadcast msg ไปยัง client ทุกคน
            #       ยกเว้น sender (conn เอง)
        else:
            disconnect(conn)
    except Exception:
        disconnect(conn)

def disconnect(conn):
    print(f'[-] Disconnected: {conn.getpeername()}')
    # TODO: ลบ conn ออกจาก clients
    # TODO: unregister + close

# TODO: setup server socket + event loop
```

ผลลัพธ์เมื่อทดลองรัน Server แล้วใช้คำสั่ง ncat 127.0.0.1 9001 ที่ Terminal ทั้ง 4 หน้าต่าง ที่ต้องการรัน Chat Client



The image shows four terminal windows arranged in a 2x2 grid. The top-left window shows a Python script running a server on port 9000, which has successfully connected to three clients. The top-right window shows a client running ncat on port 9000. The bottom-left window shows a client running ncat on port 9000. The bottom-right window shows a client running ncat on port 9000.

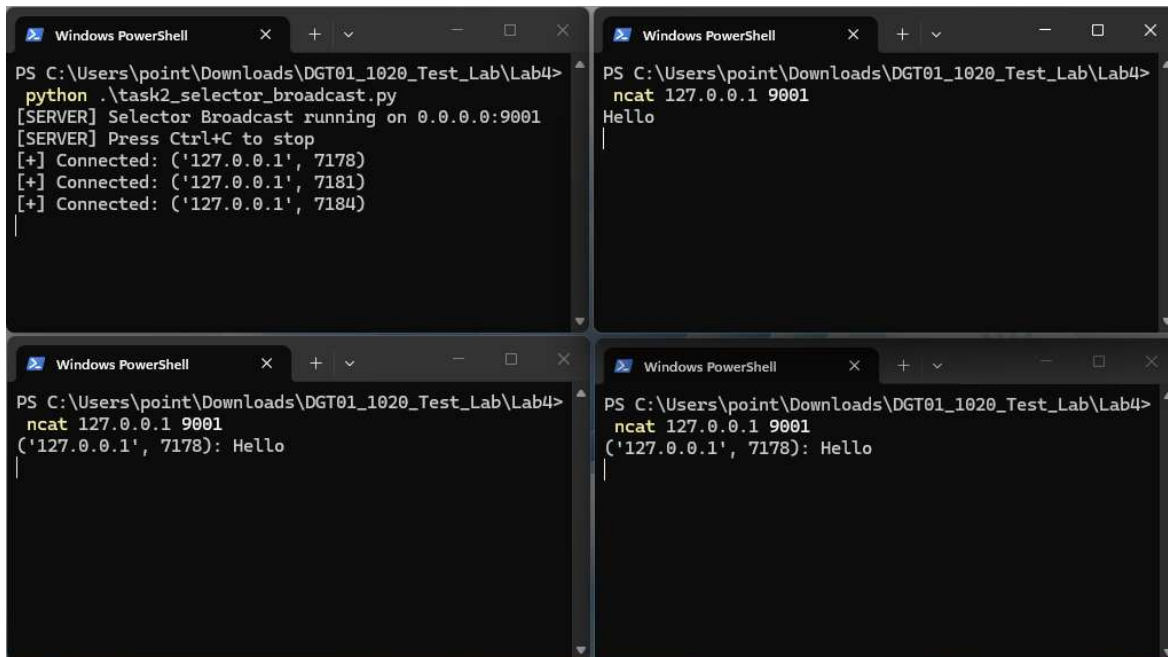
```
PS C:\Users\point\Downloads\DGT01_1020_Test_Lab\Lab4> python task1_selector_echo.py
[SERVER] Selector Echo running on 0.0.0.0:9000
[SERVER] Press Ctrl+C to stop
[+] Connected: ('127.0.0.1', 6382)
[+] Connected: ('127.0.0.1', 6385)
[+] Connected: ('127.0.0.1', 6388)

PS C:\Users\point\Downloads\DGT01_1020_Test_Lab\Lab4> ncat 127.0.0.1 9000

PS C:\Users\point\Downloads\DGT01_1020_Test_Lab\Lab4> ncat 127.0.0.1 9000

PS C:\Users\point\Downloads\DGT01_1020_Test_Lab\Lab4> ncat 127.0.0.1 9000
```

ผลลัพธ์เมื่อทดลองพิมพ์ข้อความ โดยข้อความจะไปปรากฏที่ Terminal ที่อื่นที่ไม่ใช่ที่ Terminal ที่พิมพ์ลงไป



The image shows four terminal windows arranged in a 2x2 grid. The top-left window shows a Python script running a broadcast server on port 9001, which has successfully connected to three clients. The top-right window shows a client running ncat on port 9001, which has received the message 'Hello'. The bottom-left window shows a client running ncat on port 9001, which has received the message 'Hello'. The bottom-right window shows a client running ncat on port 9001, which has received the message 'Hello'.

```
PS C:\Users\point\Downloads\DGT01_1020_Test_Lab\Lab4> python .\task2_selector_broadcast.py
[SERVER] Selector Broadcast running on 0.0.0.0:9001
[SERVER] Press Ctrl+C to stop
[+] Connected: ('127.0.0.1', 7178)
[+] Connected: ('127.0.0.1', 7181)
[+] Connected: ('127.0.0.1', 7184)

PS C:\Users\point\Downloads\DGT01_1020_Test_Lab\Lab4> ncat 127.0.0.1 9001
Hello

PS C:\Users\point\Downloads\DGT01_1020_Test_Lab\Lab4> ncat 127.0.0.1 9001
('127.0.0.1', 7178): Hello

PS C:\Users\point\Downloads\DGT01_1020_Test_Lab\Lab4> ncat 127.0.0.1 9001
('127.0.0.1', 7178): Hello
```

มีตัวอย่างผลลัพธ์ต่อในหน้าถัดไป

ผลลัพธ์เมื่อทำการพิมพ์ทุก Terminal และจากนั้นกด Ctrl + C เพื่อหยุดการทำงานทุกคำสั่งทุก Terminal

```

PS C:\Users\point\Downloads\DGT01_1020_Test_Lab\Lab4> python .\task2_selector_broadcast.py
[SERVER] Selector Broadcast running on 0.0.0.0:9001
[SERVER] Press Ctrl+C to stop
[+] Connected: ('127.0.0.1', 7178)
[+] Connected: ('127.0.0.1', 7181)
[+] Connected: ('127.0.0.1', 7184)
[-] Disconnected: ('127.0.0.1', 7184)
[-] Disconnected: ('127.0.0.1', 7181)
[-] Disconnected: ('127.0.0.1', 7178)

[SERVER] Shutting down...

PS C:\Users\point\Downloads\DGT01_1020_Test_Lab\Lab4> ncat 127.0.0.1 9001
Hello
('127.0.0.1', 7181): Hello T1.
('127.0.0.1', 7184): Hello T2.
PS C:\Users\point\Downloads\DGT01_1020_Test_Lab\Lab4>

PS C:\Users\point\Downloads\DGT01_1020_Test_Lab\Lab4> ncat 127.0.0.1 9001
('127.0.0.1', 7178): Hello
('127.0.0.1', 7181): Hello T1.
Hello T2.
PS C:\Users\point\Downloads\DGT01_1020_Test_Lab\Lab4>

```

เงื่อนไขผ่าน

- ✓ ใช้ selectors.DefaultSelector()
- ✓ มีตัวแปร clients = set() เพื่อเก็บ client socket ทั้งหมด
- ✓ เมื่อ client connect ให้เพิ่มเข้า clients
- ✓ เมื่อ client ส่งข้อความ ให้ broadcast ไปยัง client ทุกคน
- ✓ ข้อความต้องมี address ของผู้ส่ง เช่น (127.0.0.1, 51234): hello
- ✓ เมื่อ client disconnect ต้องลบออกจาก clients, unregister และ close
- ✓ ต้องมี error handling ไม่ให้ event loop พัง
- ✓ ห้ามใช้ thread และห้ามใช้ Lock

Note: ทำไมไม่ต้องการ Lock? — selectors รันบน 1 thread → ไม่มี concurrent access บน clients set จึงไม่มี race condition

แบบฝึกหัด

Asyncio Echo Server

3

เวลา: 35 นาที | Skill: async/await, asyncio.start_server, reader/writer

โจทย์: แปลงแนวคิดจาก selectors เป็น asyncio Streams เพื่อเขียน concurrent server ที่อ่านเหมือน sequential code มากขึ้น

ตัวอย่างโค้ดที่เตรียมไว้ให้

Skeleton — task3_asyncio_echo.py

```
# task3_asyncio_echo.py
import asyncio

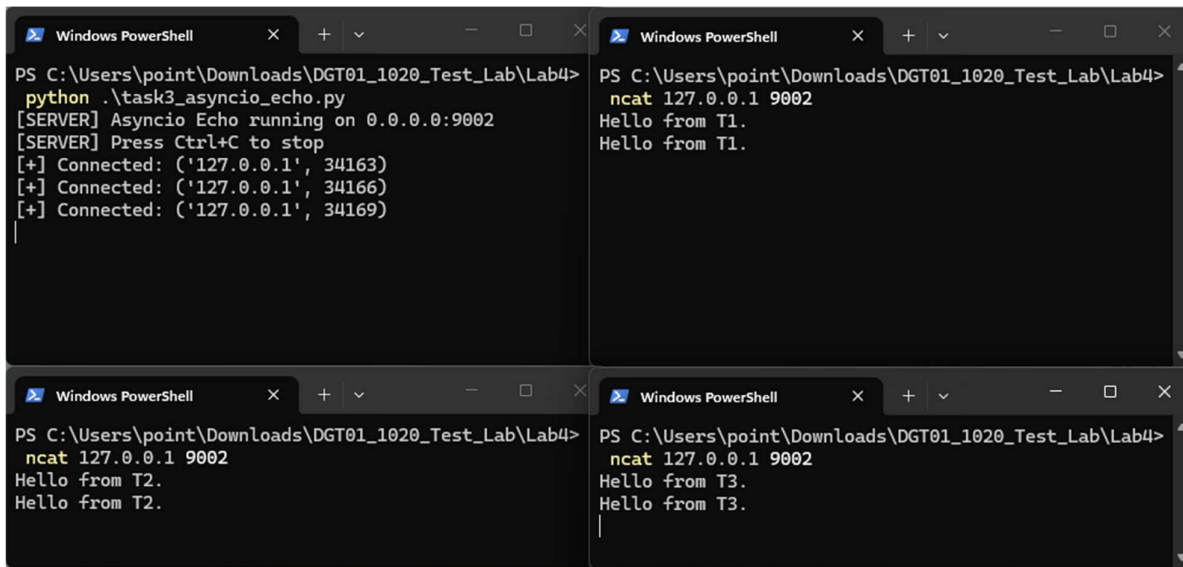
HOST = '0.0.0.0'
PORT = 9002

async def handle_client(reader, writer):
    addr = writer.get_extra_info('peername')
    print(f'[+] Connected: {addr}')
    try:
        while True:
            # TODO: อ่าน data จาก reader (1024 bytes)
            # TODO: ถ้า data ว่าง → break
            # TODO: เขียน data กลับไป writer
            # TODO: await writer.drain()
            pass
    except Exception as e:
        print(f'[-] Error: {e}')
    finally:
        print(f'[-] Disconnected: {addr}')
        # TODO: ปิด writer + await wait_closed()

async def main():
    server = await asyncio.start_server(
        handle_client, HOST, PORT)
    print(f'[SERVER] Asyncio Echo running on {HOST}:{PORT}')
    async with server:
        await server.serve_forever()

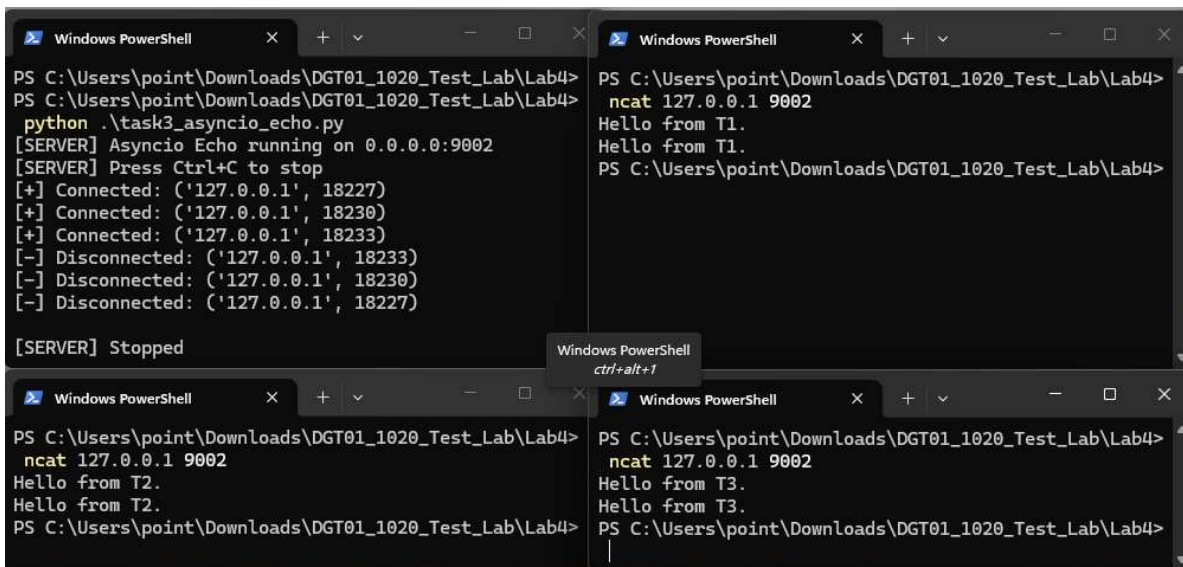
asyncio.run(main())
```

ผลลัพธ์เมื่อทดลองรัน Server แล้วใช้คำสั่ง `ncat 127.0.0.1 9002` ที่ Terminal ทั้ง 4 หน้าต่าง ที่ต้องการรัน Chat Client โดยเมื่อพิมพ์ข้อความลงในแต่ละ Terminal แล้ว จะปรากฏข้อความที่พิมพ์ลงไป Reply กลับมาเป็นข้อความที่พิมพ์ไปก่อนหน้านี้



The image shows four terminal windows arranged in a 2x2 grid. The top-left window shows the server running a Python script `task3_asyncio_echo.py`. It displays messages for three connected clients: '127.0.0.1', '127.0.0.1', and '127.0.0.1' with ports 34163, 34166, and 34169 respectively. The top-right window shows a client terminal where the command `ncat 127.0.0.1 9002` has been entered, and it has received two 'Hello from T1.' messages. The bottom-left window shows another client terminal where the command `ncat 127.0.0.1 9002` has been entered, and it has received two 'Hello from T2.' messages. The bottom-right window shows a third client terminal where the command `ncat 127.0.0.1 9002` has been entered, and it has received two 'Hello from T3.' messages.

ผลลัพธ์เมื่อทำการพิมพ์ทุก Terminal และจากนั้นกด `Ctrl + C` เพื่อหยุดการทำงานทุกคำสั่งทุก Terminal



The image shows four terminal windows arranged in a 2x2 grid. The top-left window shows the server running the Python script. It displays messages for three connected clients: '127.0.0.1', '127.0.0.1', and '127.0.0.1' with ports 18227, 18230, and 18233 respectively. It also shows disconnection messages for each client. The top-right window shows a client terminal where the command `ncat 127.0.0.1 9002` has been entered, and it has received two 'Hello from T1.' messages. The bottom-left window shows another client terminal where the command `ncat 127.0.0.1 9002` has been entered, and it has received two 'Hello from T2.' messages. The bottom-right window shows a third client terminal where the command `ncat 127.0.0.1 9002` has been entered, and it has received two 'Hello from T3.' messages. A small tooltip 'Windows PowerShell ctrl+alt+1' is visible over the top-right window.

คำเตือน

`asyncio` ≠ parallel multi-core — เป็น concurrency ใน 1 thread เดียว ห้ามใช้ `time.sleep()` หรือ `requests.get()` ใน `async` function เพราะจะ block ทั้ง event loop

เงื่อนไขผ่าน

- ✓ ใช้ `asyncio.start_server()`
- ✓ มี `async def handle_client(reader, writer)`
- ✓ ใช้ `await reader.read(1024)` อ่านข้อมูล
- ✓ ใช้ `writer.write(data)` และ `await writer.drain()`
- ✓ ปิด connection ด้วย `writer.close()` และ `await writer.wait_closed()`
- ✓ รองรับหลาย client พร้อมกัน
- ✓ ห้ามใช้ `selectors`, `threading` หรือ `ThreadPoolExecutor`

แบบฝึกหัด

4

Asyncio Chat Server

เวลา: 40 นาที | Skill: asyncio shared state, broadcast, nickname

โจทย์: สร้าง asyncio chat server ที่รองรับหลาย client โดย client แต่ละคนต้องส่ง nickname ก่อน แล้ว server จะ broadcast ข้อความระหว่างกัน

Requirements

R	Feature
R1	client connect แล้วต้องส่ง nickname ก่อน เช่น Alice
R2	server broadcast 'Alice joined' ให้ทุกคน เมื่อ client เข้ามา
R3	ข้อความ chat broadcast ในรูปแบบ Alice: hello
R4	ใช้ dict clients = {writer: nickname} เก็บ state — ห้ามใช้ Lock
R5	เมื่อ client disconnect ต้อง broadcast 'Alice left'
R6	รองรับอย่างน้อย 5 clients พร้อมกัน

ตัวอย่างโค้ดที่เตรียมไว้ให้

Skeleton — task4_asyncio_chat.py

```
# task4_asyncio_chat.py
import asyncio

HOST = '0.0.0.0'
PORT = 9003

clients = {} # writer -> nickname

async def broadcast(message, sender=None):
    dead_clients = []
    for writer in clients:
        if writer is sender:
            continue
        try:
            writer.write(message.encode())
            await writer.drain()
        except Exception:
            dead_clients.append(writer)
```

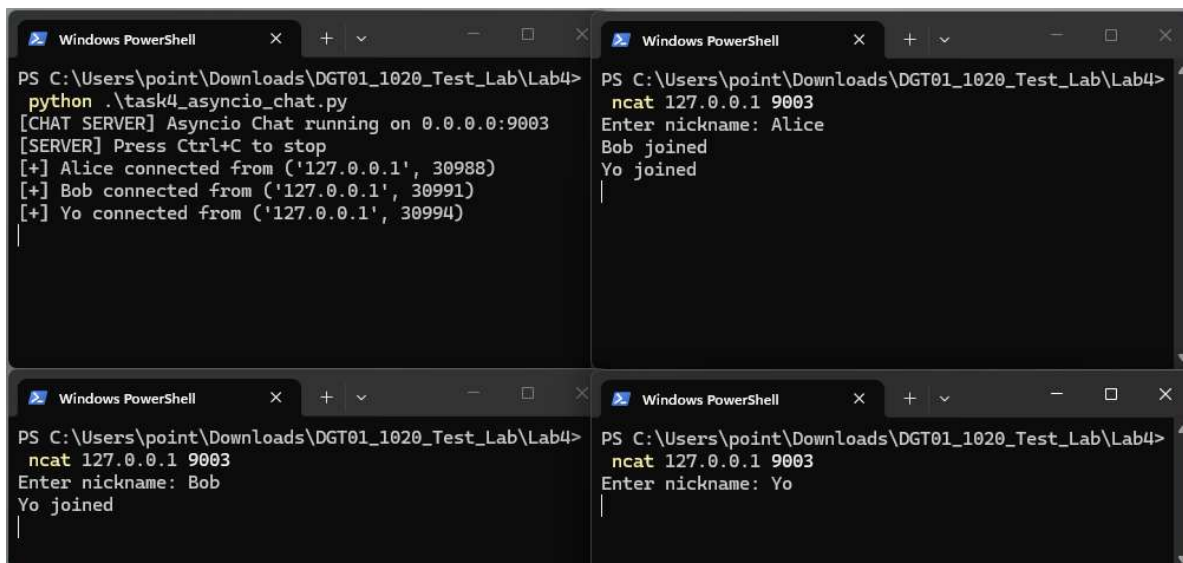
```
# TODO: cleanup dead clients

async def handle_client(reader, writer):
    addr = writer.get_extra_info('peername')
    # TODO: รับ nickname จาก client
    # TODO: เพิ่มเข้า clients dict
    # TODO: broadcast 'nickname joined'
    try:
        while True:
            data = await reader.read(1024)
            if not data: break
            # TODO: broadcast f'{nickname}: {data.decode()}'
    finally:
        if writer in clients:
            nickname = clients[writer]
            del clients[writer]
            # TODO: broadcast 'nickname left'
            writer.close()
            await writer.wait_closed()

async def main():
    server = await asyncio.start_server(
        handle_client, HOST, PORT)
    async with server:
        await server.serve_forever()

asyncio.run(main())
```

ผลลัพธ์การรัน: เมื่อทำการรัน Server และ Client ที่ได้ทำการตั้งชื่อในแต่ Client ในแต่ละ Terminal ได้เรียบร้อยแล้วโดยที่ Terminal ที่รัน Server จะแสดงว่าใครเข้าเชื่อมต่อแล้วบ้าง และที่ Terminal ของ Client ที่ไม่ทำการพิมพ์ข้อความลงไปจะปรากฏว่าใครเข้า Join ใน Server บ้าง



The image shows four Windows PowerShell terminal windows arranged in a 2x2 grid. The top-left window shows the server running with the command `python .\task4_asyncio_chat.py`. It displays messages for three clients: Alice, Bob, and Yo, showing their connection status and IP addresses. The top-right window shows a client running `ncat 127.0.0.1 9003`, entering the nickname 'Alice', and seeing 'Bob joined' and 'Yo joined' in the output. The bottom-left window shows another client running `ncat 127.0.0.1 9003`, entering the nickname 'Bob', and seeing 'Yo joined'. The bottom-right window shows a third client running `ncat 127.0.0.1 9003`, entering the nickname 'Yo', and seeing no further output.

ผลลัพธ์การรัน: เมื่อทำการสนทนากันเกิดขึ้น จะเป็นการแสดงถึงข้อความที่แต่ละ Client พิมพ์ โดยจะแสดงชื่อและข้อความที่พิมพ์เข้ามาที่ Server และจะปรากฏข้อความที่พิมพ์ในแต่ละ Client ที่ทำการเชื่อมต่อเข้ามา จากนั้นหากมีใครกด Ctrl + C เพื่อออกไปแล้วนั้น จะปรากฏชื่อและข้อความ “\${NAME} disconnected” ออกจากการสนทนา

```

PS C:\Users\point\Downloads\DGT01_1020_Test_Lab\Lab4> python .\task4_asyncio_chat.py
[CHAT SERVER] Asyncio Chat running on 0.0.0.0:9003
[SERVER] Press Ctrl+C to stop
[+] Alice connected from ('127.0.0.1', 30988)
[+] Bob connected from ('127.0.0.1', 30991)
[+] Yo connected from ('127.0.0.1', 30994)
[-] Yo disconnected
[-] Bob disconnected
[-] Alice disconnected

PS C:\Users\point\Downloads\DGT01_1020_Test_Lab\Lab4> ncat 127.0.0.1 9003
Enter nickname: Alice
Bob joined
Yo joined
Hello everyone!
Bob: Hi guy.
Yo: Hi!
Yo left
Bob left
PS C:\Users\point\Downloads\DGT01_1020_Test_Lab\Lab4>

PS C:\Users\point\Downloads\DGT01_1020_Test_Lab\Lab4> ncat 127.0.0.1 9003
Enter nickname: Bob
Yo joined
Alice: Hello everyone!
Hi guy.
Yo: Hi!
Yo left
PS C:\Users\point\Downloads\DGT01_1020_Test_Lab\Lab4>

PS C:\Users\point\Downloads\DGT01_1020_Test_Lab\Lab4> ncat 127.0.0.1 9003
Enter nickname: Yo
Alice: Hello everyone!
Bob: Hi guy.
Hi!
PS C:\Users\point\Downloads\DGT01_1020_Test_Lab\Lab4>

```

เงื่อนไขผ่าน

- ✓ R1: client ส่ง nickname ก่อนเริ่ม chat
- ✓ R2: broadcast '<nickname> joined' เมื่อมีคนเข้า
- ✓ R3: ข้อความรูปแบบ <nickname>: <message>
- ✓ R4: ใช้ dict clients = {writer: nickname} โดยไม่ต้องใช้ Lock
- ✓ R5: broadcast '<nickname> left' เมื่อมีคนออก
- ✓ R6: ทดสอบ nc 5 หน้าต่างพร้อมกันได้สำเร็จ

Hint: ทำไมไม่ต้องการ Lock? — coroutine จะ yield control เฉพาะตอน await เท่านั้น ระหว่างที่แก้ clients dict ไม่มี await จึงไม่มี coroutine อื่นมา interrupt

การส่งงาน

ไฟล์ที่ต้องส่ง	task1_selector_echo.py, task2_selector_broadcast.py, task3_asyncio_echo.py, task4_asyncio_chat.py
กำหนดส่ง	ก่อนสิ้นสุดคาบ Lab ก่อนเวลา 16.40 น.
คะแนนรวม	100% (Task 1: 25% Task 2: 25% Task 3: 25% Task 4: 25%)